First, I started off by labeling the database as Online Music Store. Then I created the five tables: artists, songs, customers, purchases and streams. Afterwards I uploaded the five csv files.

1) **Basic Selection: Retrieve the titles and release dates of all songs released in 2022, ordered by release date (newest first).**

SELECT title,release_date **FROM** Songs **WHERE** release_date >='2022-01-01' **AND** release_date < '2023-01-01'

ORDER BY release_date **DESC**

2) **Filtering: Find all songs with a popularity score greater than 80 and a duration less than 4 minutes (240 seconds).**

SELECT* FROM songs **WHERE** popularity_score > 80 **AND** duration_seconds <240;

3) **Pattern Matching**: List all artists whose names start with "The".

SELECT * FROM artists **WHERE** artist_name **LIKE** 'The%'

4) **Multiple Conditions**: Find all premium customers who joined in 2022.

SELECT * FROM customers **WHERE** premium_member= TRUE **AND** join_date **>= '2022-01-01' AND join_date < '2023-01-01';**

5) **Calculations and Aliasing**: Calculate the total duration (in minutes) of all songs in the database and display the result with an appropriate column name.

SELECT SUM(duration_seconds)/60.0 **AS** total_duration_minutes **FROM** songs;

6) **Advanced Filtering**: Find the top 5 most expensive song purchases in the database.

SELECT * FROM purchases **ORDER BY** price **DESC**
LIMIT 5;

7) Using Multiple Tables Separately: First, find all song_ids from songs with a popularity score above 90. Then, use those song_ids to find purchases of those songs.

```
SELECT song_id FROM songs WHERE popularity_score>90;

SELECT * FROM purchases WHERE song_id in (
SELECT song_id
FROM songs
WHERE popularity_score>90
);
```

8) **Range Checking:** Find all purchases made between January 1, 2023 and March 31, 2023.

```
SELECT *
FROM purchases
WHERE purchase_date BETWEEN '2023-01-01' AND '2023-03-31';
```

9) **Advanced Filtering** with ORDER BY: Identify the songs with the highest popularity scores (above 90).

```
SELECT *
FROM songs
WHERE popularity_score>90
ORDER BY popularity_score DESC;
```

Discussion Questions

1. **In our database design, we separated purchases and streams into different tables. What are the benefits of this approach versus having a single "user_interactions" table?**

The benefits of this approach versus having a single ‘user_interactions’ table is that it leads to better organization, accounts for how purchases and streams require different information, improved data accuracy, easier reporting and analysis and improved data integrity.

2. **Based on the provided data model, what business questions could music executives answer using SQL queries that we haven't covered in our exercise?**

Some business questions that music executives answer using SQL queries include the following:

- What songs generate the highest revenue?
- Which artists generate the most money from purchases?
- What is the average amount that customers spend on music?
- Which customers purchase the most songs?
- Which customers have stopped purchasing music
- Which artists have the most popular songs?

3. **How would you extend this schema to track more detailed user behavior, such as when users skip songs or how much of a song they listen to before skipping?**

There can be new tables titled listening_history or user_interactions

4. **For tasks 1-3, how could you combine them into a single, more complex query that finds popular short songs by artists whose names start with "The"**

This can be combined by using JOIN, WHERE and ORDER BY